Análisis comparativo: Minimax vs. Árbol de Decisión en Tres en Raya

Victor David Sosa Coca · Estructuras de Datos II · 20/10/2026

1. Resumen

Este informe compara dos agentes que juegan Tres en Raya como jugador O: el agente **Minimax** con Backtracking, que explora el árbol de jugadas en tiempo real, y un **árbol de decisión** (DecisionTreeClassifier de Scikit-Learn) entrenado para imitar las jugadas de Minimax. Se midió la velocidad de cada uno, la precisión de las jugadas del árbol y su desempeño en partidas completas.

La conclusión principal es que Minimax es exacto y, en este juego tan pequeño, casi siempre igual de rápido o más. El árbol de decisión ofrece una latencia estable y mucho menor en la primera jugada, a cambio de equivocarse en parte de los tableros que no vio al entrenar.

2. Metodología

- **Tableros evaluados:** los 2,097 tableros posibles en los que le toca jugar a O (X mueve primero y la partida no ha terminado).
- **Minimax:** puntaje +10 por victoria de la IA y -10 por derrota, restando la profundidad para preferir victorias rápidas y retrasar derrotas.
- **Árbol de decisión:** entrenado con un dataset de 390 tableros distintos, generado simulando 1,000 partidas entre Minimax (O) y un jugador aleatorio (X), con semilla 42. Las 9 casillas se codifican como 0 (vacía), 1 (X) y -1 (O), y la etiqueta es la casilla que elige Minimax.
- **Velocidad:** se mide el mejor de 3 intentos por jugada con `time.perf_counter`. Equipo utilizado: AMD Ryzen 7 6800 H, memoria ram 16 GB DDR5, Python 3.14.
- **Precisión:** se usan tres métricas. *Igual a Minimax* exige la misma casilla. *Óptima* exige la casilla de mejor puntaje exacto. *No empeora* exige conservar el resultado teórico (ganar, empatar o perder igual que con Minimax).
- **Partidas:** 1,000 por enfrentamiento, con el agente jugando como O contra un rival aleatorio y contra un rival perfecto (Minimax que desempata al azar). Ambos agentes enfrentan las mismas secuencias aleatorias.

3. Velocidad

Medida	Minimax	Árbol ML
Promedio (ms)	1.61	1.94
Mediana (ms)	0.13	1.78
Máximo (ms)	316.24	5.64
Nodos explorados (promedio)	415	-

Por momento de la partida:

Jugada de O	Tableros	Minimax (ms)	Árbol (ms)	Nodos Minimax
1.ª	9	223.51	2.41	61,104
2.ª	252	4.56	2.12	1,090
3.ª	1,140	0.18	1.93	39
4.ª	696	0.02	1.87	3

El costo de Minimax depende de cuántas casillas quedan libres: en la primera jugada explora 61,104 nodos y tarda unos 224 ms, pero en la tercera y cuarta jugada casi no tiene que buscar y responde en menos de 0.2 ms. El árbol, en cambio, tarda entre 1.9 y 2.4 ms siempre, porque su costo no depende del tablero.

Por eso el árbol es unas 93 veces más rápido en la primera jugada, pero más lento que Minimax en la tercera y cuarta. Sumando los promedios de las cuatro jugadas de O, una partida completa cuesta aproximadamente 228 ms con Minimax y 8 ms con el árbol, aunque casi todo el tiempo de Minimax se concentra en la primera jugada. La ventaja más clara del árbol es el peor caso: su máximo es 5.64 ms frente a 316.24 ms.

Es importante notar que la mayor parte del tiempo del árbol no es recorrer el árbol, sino la sobrecarga de pandas y Scikit-Learn al preparar la entrada. Midiendo solo la predicción con un arreglo de numpy, el promedio baja a 0.314 ms por jugada.

4. Precisión de las jugadas del árbol

Grupo de tableros	Tableros	Igual a Minimax	Óptima	No empeora
Todos	2,097	62%	75%	80%

Grupo de tableros	Tableros	Igual a Minimax	Óptima	No empeora
No vistos al entrenar	1,707	53%	69%	75%
De entrenamiento	390	100%	100%	100%

El 100% en los tableros de entrenamiento no mide aprendizaje: el árbol no tiene límite de profundidad y memoriza esos casos. La cifra relevante es la de los **tableros no vistos**: el árbol coincide con Minimax en el 53%, elige una jugada óptima en el 69% y conserva el resultado teórico en el 75%. En otras palabras, en uno de cada cuatro tableros nuevos su jugada empeora el resultado que se podía lograr.

Además, en alrededor del 13% de los tableros no vistos el árbol predice una casilla que ya está ocupada. En ese caso el agente juega la primera casilla libre, lo que sirve de respaldo pero no es una jugada pensada.

5. Resultados en partidas

Rival (X)	Agente	Gana	Empata	Pierde
Aleatorio	Minimax	823	177	0
Aleatorio	Árbol ML	817	171	12
Perfecto	Minimax	0	1,000	0
Perfecto	Árbol ML	0	991	9

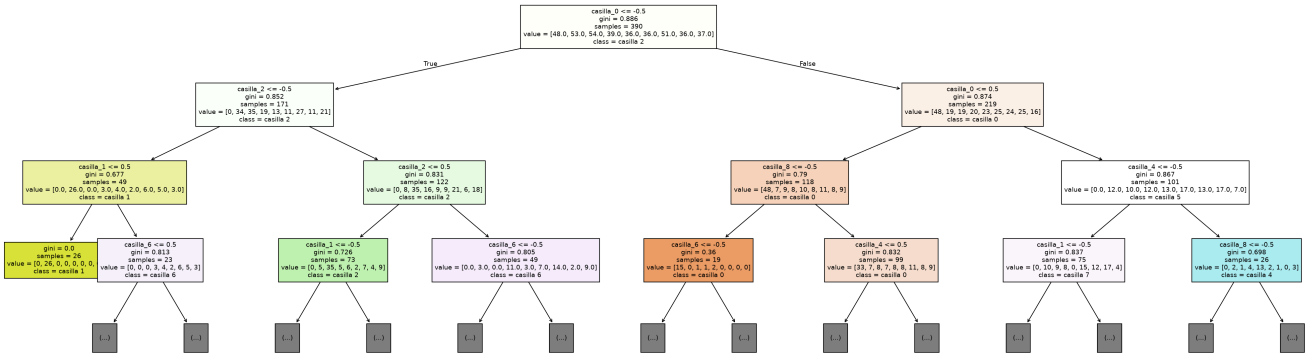
Minimax no perdió ninguna de las 2,000 partidas, lo que confirma que es un agente imbatible. El árbol, pese a coincidir con Minimax solo en poco más de la mitad de los tableros nuevos, pierde únicamente el 1.2% de las partidas contra el rival aleatorio y el 0.9% contra el perfecto. Una explicación probable es que muchos tableros admiten varias jugadas igual de buenas y que las partidas reales visitan sobre todo los tableros más frecuentes, que están mejor cubiertos por el dataset. Aun así, sus derrotas muestran que no es infalible.

6. El modelo entrenado

- Tableros en el dataset: 390, es decir, cerca del 19% de los 2,097 tableros posibles.
- Profundidad del árbol: 14.
- Hojas del árbol: 165.

- Validación cruzada (5 grupos, misma casilla que Minimax, sobre tableros no vistos): promedio 57.2%, mínimo 50.0% y máximo 61.5%.

Gráfico del árbol de decisión entrenado (solo se dibujan los primeros niveles):



Fragmento de las reglas aprendidas (las completas están en `data/reglas_arbol.txt`). Cada casilla vale 1 si tiene X, -1 si tiene O y 0 si está vacía, y `class` indica la casilla donde juega el agente:

```
|--- casilla_0 <= -0.50
|   |--- casilla_2 <= -0.50
|   |   |--- casilla_1 <= 0.50
|   |   |   |--- class: 1
|   |   |--- casilla_1 >  0.50
|   |   |   |--- casilla_6 <= 0.50
|   |   |   |   |--- casilla_4 <= -0.50
|   |   |   |   |   |--- class: 6
```

Este fragmento contiene dos reglas que coinciden con decisiones óptimas de Minimax. Si hay una O en las casillas 0 y 2 y la casilla 1 no tiene una X, el agente juega en la 1 y completa la fila superior. Si la casilla 1 tiene una X, la 4 tiene una O y la 6 no tiene una X, juega en la 6 y completa la diagonal 2-4-6. El árbol llegó a estas reglas solo a partir de los datos, sin recorrer el árbol de jugadas.

7. Discusión: tiempo de respuesta frente a precisión

Minimax garantiza la jugada óptima sin necesitar datos, pero su costo crece con el tamaño del árbol de búsqueda. En Tres en Raya ese costo solo es notable en la primera jugada, así que en la práctica es rápido casi siempre.

El árbol de decisión convierte la búsqueda en una consulta de latencia casi constante, pero solo aproxima la decisión de Minimax: depende de la calidad y cobertura de sus datos y puede equivocarse o proponer jugadas ilegales.

En un juego tan pequeño, el aprendizaje automático no supera a la búsqueda exhaustiva. Su valor aparece en problemas donde explorar todo el árbol es inviable, como el ajedrez o el Go: allí aproximar la función de decisión con un modelo entrenado permite responder rápido sin buscar. Este proyecto muestra ese mecanismo a escala pequeña.

8. Limitaciones

- El agente de ML solo juega como O, con X moviendo primero, igual que en los datos de entrenamiento.
- El dataset cubre cerca del 19% de los tableros posibles, y al generarse contra un rival aleatorio no incluye bien las posiciones poco probables.
- Las métricas de tableros no vistos pesan por igual los 1,707 tableros, aunque en partidas reales algunos aparecen mucho más que otros.
- El árbol se entrenó sin límite de profundidad ni poda, lo que favorece el sobreajuste (100% en entrenamiento frente a 53% en tableros no vistos).
- Los tiempos provienen de una sola máquina y dependen del equipo.

9. Conclusiones y mejoras posibles

1. Minimax es el agente más preciso y el único imbatible. El árbol de decisión es más predecible en latencia, pero comete errores.
2. La diferencia de velocidad depende de la jugada: el árbol gana en la primera y Minimax en las últimas.
3. Mejoras posibles: simular más partidas para ampliar el dataset, limitar la profundidad del árbol o aplicar poda para reducir el sobreajuste, usar numpy en la predicción para bajar la sobrecarga, y como desafío adicional implementar poda alfa-beta en Minimax.